

SUPSI

Sviluppo di un Agente AI Autonomo in Java Spring Boot

Studente/i

Pasquini Mattia

Relatore

Besenzoni Matteo

Correlatore

-

Committente

Besenzoni Matteo

Corso di laurea

Ingegneria informatica TP

Codice progetto

C11308

Anno

2025/2026

Data

26 agosto 2026

STUDENTSUPSI

Ringraziamenti

Ringrazio al mio relatore, Besenzoni, per la sua disponibilità e l'assistenza durante lo svolgimento di questo progetto.

Ringrazio anche alla mia famiglia per il loro sostegno morale durante questo lavoro.

Ringrazio anche me stesso per aver affrontato questo lavoro con impegno e determinazione, senza mai mollare.

Indice

1	Introduzione	1
2	Motivazione e contesto	3
3	Stato dell'arte	5
3.1	Large Language Model	5
3.2	Agente AI	5
3.2.1	Pattern ReAct	6
3.2.2	Tool calling	7
3.3	Sicurezza	8
3.3.1	Prompt Injection	8
3.3.1.1	Direct prompt injection	8
3.3.1.2	Indirect prompt injection	8
3.3.2	Path Traversal	8
3.3.3	Mitigazione	8
3.4	Human-in-the-loop	9
3.5	Google Gen AI SDK	9
3.6	Soluzioni esistenti	9
4	Analisi	11
4.1	Requisiti funzionali e non funzionali	11
4.1.1	Requisiti funzionali	11
4.1.2	Requisiti non funzionali	13
4.2	Use case	14
5	Progettazione	15
5.1	Pianificazione	15
5.2	Gestione del progetto	15
5.3	Tecnologie	16
5.3.1	Backend	16
5.3.2	Frontend	16

5.3.3	Version control	16
5.4	Architettura	17
5.5	Struttura del progetto	18
5.5.1	Frontend	18
5.5.2	Backend	18
5.6	Scelta del modello LLM di Gemini	18
5.7	Design delle interfacce	19
5.8	Diagrammi	20
5.8.1	Diagramma di flusso di gestione di function calling	20
5.8.2	Diagramma di sequenza di un function calling	21
5.8.3	Diagramma di sequenza di un function calling con human in the loop	22
6	Implementazione	23
6.1	Configurazione Spring Boot	23
6.2	Endpoint REST	24
6.3	Service	24
6.4	Sicurezza	24
6.5	Tools	25
6.5.1	Annotation	25
6.5.2	Implementazione dei tools	25
6.6	Scansione e registro dei tools	28
6.7	Invocazione a runtime	28
6.8	Configurazione della sessione chat stateful	28
6.9	Meccanismo di function calling	29
6.10	Gestione degli errori	29
6.10.1	HTTP	29
6.10.2	Livello agentic	30
6.11	Logging	30
6.12	Frontend	31
7	Test	33
7.1	Metodologia di testing	33
7.2	Strumenti utilizzati	33
7.3	Unit testing	33
7.4	Risultati dei test	33
7.5	Pipeline CI/CD	34
7.5.1	Struttura della pipeline	34

8	Conclusioni	35
8.1	Risultati	35
8.2	Sviluppi futuri	35
8.2.1	Nuovi tool	35
8.2.2	Conversazioni persistenti	35
8.2.3	Supporto multi-provider LLM	36
8.2.4	Streaming delle risposte in tempo reale	36

Elenco delle figure

3.1	Flusso ReAct	6
3.2	Flusso di un singolo tool calling	7
4.1	Diagramma use case	14
5.1	Gantt per la pianificazione temporale preventivo	15
5.2	Architettura dell'applicativo	17
5.3	Mockup della chatbot	19
5.4	Diagramma di flusso di gestione di function calling	20
5.5	Diagramma di sequenza di un function call	21
5.6	Diagramma di sequenza di un function call con human-in-the-loop accettando l'invocazione	22

Elenco delle tabelle

5.1 Limiti dei modelli Gemini nel piano gratuito di Google AI Studio 19

6.1 API REST del backend 24

6.2 Metodi invocabili dall'agente AI 27

Abstract

Italiano

Questo progetto di diploma riguarda lo sviluppo di un agente di intelligenza artificiale autonomo, realizzato in Java con Spring Boot, in grado di interpretare richieste espresse in linguaggio naturale ed eseguire operazioni sul file system locale. L'agente utilizza un Large Language Model di Gemini e il meccanismo di function calling per interagire con un insieme di strumenti dedicati alla gestione di file e directory. Il pattern ReAct permette al sistema di eseguire iterativamente più operazioni, utilizzando i risultati ottenuti precedentemente per proseguire autonomamente verso il raggiungimento dell'obiettivo richiesto. Particolare attenzione è stata dedicata alla sicurezza, limitando tutte le operazioni a una directory sandbox e implementando controlli contro attacchi di tipo path traversal. Per le operazioni distruttive è stato inoltre introdotto un meccanismo human-in-the-loop, che richiede la conferma esplicita dell'utente prima dell'esecuzione. Il sistema è dotato di un'interfaccia web per l'interazione con l'agente. La soluzione implementata soddisfa gli obiettivi e i requisiti definiti, permettendo di gestire il file system attraverso comandi in linguaggio naturale mantenendo un adeguato livello di controllo e isolamento.

English

This diploma project focuses on the development of an autonomous artificial intelligence agent, implemented in Java using Spring Boot, capable of interpreting natural language requests and performing operations on the local file system. The agent relies on a Gemini Large Language Model and the function calling mechanism to interact with a set of dedicated tools for managing files and directories. The ReAct pattern allows the system to iteratively perform multiple operations, using the results obtained from previous actions to autonomously progress towards the achievement of the requested goal.

Particular attention is also paid to security by restricting all operations to a sandbox directory and implementing protection mechanisms against path traversal attacks. For destructive operations, a human-in-the-loop mechanism has been introduced, requiring expli-

cit user confirmation before execution. The system is also equipped with a web interface for interacting with the agent.

The implemented solution meets the defined objectives and requirements, enabling the file system to be managed through natural language commands while maintaining an adequate level of control and isolation.

L'utilizzo dell'AI

Durante lo svolgimento del progetto è stata utilizzata l'AI generativa. In particolare, essa è stata impiegata nella fase di documentazione come supporto sintattico e linguistico nella stesura della tesi. Parzialmente è stata utilizzata anche durante la fase dello sviluppo, sempre sotto la mia supervisione. Le scelte progettuali, di design e di architettura del sistema sono state tuttavia elaborate personalmente.

Capitolo 1

Introduzione

Questa tesi si concentra sullo sviluppo di un'applicazione Java basata su Spring Boot, progettata per fungere da agente intelligente per la gestione del file system locale. L'obiettivo è realizzare un sistema in grado di interpretare comandi complessi espressi in linguaggio naturale, come ad esempio: "Trova tutti i file .log, riassumi gli errori e sposta i file in una cartella di archivio" ed eseguirli autonomamente. Inoltre, l'agente deve operare all'interno di uno spazio di lavoro limitato, senza avere la possibilità di accedere a file o cartelle (directory) esterni ad esso.

Il documento inizia illustrando la motivazione e il contesto che hanno portato allo sviluppo del progetto.

Nel capitolo successivo viene analizzato lo stato dell'arte, studiando le tecnologie e le soluzioni attualmente disponibili che possono essere utilizzate per la realizzazione dell'applicazione.

Successivamente, nel capitolo *Analisi* vengono illustrate le attività di analisi svolte, individuando le problematiche da affrontare e definendo i requisiti necessari per identificare le funzionalità da implementare.

Nel capitolo *Progettazione* vengono mostrate le scelte architetture e progettuali adottate, le tecnologie selezionate e gli aspetti più rilevanti presi in considerazione durante questa fase di lavoro.

Nel capitolo *Implementazione* viene descritto lo sviluppo del sistema e delle funzionalità previste, mentre nel capitolo *Test* vengono presentate le tipologie di testing adottate per verificare il corretto funzionamento del codice realizzato.

Infine, nel capitolo *Conclusioni* vengono analizzati i risultati finali del progetto e viene effettuato un confronto con gli obiettivi inizialmente prefissati. Vengono inoltre discussi i possibili sviluppi futuri, con l'obiettivo di migliorare il sistema e arricchirlo con nuove funzionalità.

Capitolo 2

Motivazione e contesto

Questo lavoro viene svolto come progetto di diploma al termine dell'ultimo anno di Ingegneria Informatica presso il Dipartimento Tecnologie Innovative della SUPSI. Questo progetto, proposto da un committente, è stato scelto e realizzato dal sottoscritto, con il supporto di un relatore.

Il progetto di diploma rappresenta un'occasione per dimostrare e mettere in pratica le competenze acquisite durante il percorso formativo e, allo stesso tempo, offre la possibilità di approfondire e sviluppare nuove conoscenze. Infatti, il settore tecnologico è in continua evoluzione, con l'introduzione costante di nuove tecnologie e soluzioni.

La scelta di questo progetto nasce dalla curiosità verso lo sviluppo di un agente basato sull'intelligenza artificiale e sull'integrazione di un LLM (Large Language Model). Infatti, durante il percorso formativo sono stati realizzati diversi progetti senza avere l'opportunità di approfondire e implementare tecnologie di questo tipo. Questo lavoro rappresenta quindi un'opportunità per acquisire nuove competenze in questo ambito, approfondirle e comprenderne le potenzialità.

Capitolo 3

Stato dell'arte

3.1 Large Language Model

Gli LLMs (*Large Language Models*) sono i modelli di intelligenza artificiale addestrati su una grande quantità di testo per poter comprendere, elaborare e generare testo in linguaggio naturale. [1]

I modelli di intelligenza artificiale utilizzano i token, che rappresentano le unità in cui viene suddiviso il testo perché gli LLM non elaborano direttamente le parole, ma le sequenze di token. Un token può rappresentare una parola intera, una parte di una parola o un carattere. [1]

Gli LLMs imparano osservando i pattern presenti nei testi e riconoscono le relazioni tra i diversi token. Quando un LLM riceve un input sotto forma di testo, utilizzano le informazioni apprese durante l'addestramento per calcolare la probabilità dei possibili token successivi. In questo modo, il modello seleziona un token alla volta, secondo le probabilità calcolate dal modello, e tale processo viene ripetuto più volte fino a ottenere l'output finale. [1]

Inoltre, i modelli hanno una context window, ovvero una finestra di contesto che definisce il numero massimo di token che un modello può utilizzare durante l'elaborazione di una richiesta. I modelli più recenti possono supportare finestre di contesto di centinaia di migliaia di token, consentendo di elaborare grandi quantità di testo e di gestire le conversazioni lunghe o le grandi quantità di documenti. [1]

Grazie a queste capacità, gli LLM possono essere utilizzati per svolgere diverse attività di elaborazione del linguaggio naturale, tra cui la generazione e il riassunto dei testi, l'analisi dei contenuti, la traduzione automatica e la generazione del codice.

3.2 Agente AI

Un agente AI è un sistema in grado di pianificare autonomamente le azioni necessarie per raggiungere gli obiettivi espressi in linguaggio naturale, eseguirle interagendo con strumenti

esterni e adattare il proprio comportamento in base ai risultati ottenuti con un intervento umano minimo [2].

A differenza di utilizzare solo un LLM, che si limita a generare testo in risposta a un prompt, un agente AI compie le azioni concrete nel mondo digitale, ad esempio, può interrogare un database, chiamare un'API REST, scrivere un file e usare il risultato di queste azioni per decidere il passo successivo [3]. Quindi un agente AI è un sistema che, in base al contesto e in base alla richiesta scritta in linguaggio naturale, pianifica una sequenza di azioni per raggiungere un obiettivo e le esegue utilizzando i vari strumenti disponibili, iterando fino a quando l'obiettivo non è stato raggiunto.

3.2.1 Pattern ReAct

ReAct (Reasoning and Acting) è uno dei pattern di ragionamento che permette di risolvere i problemi anche complessi e raggiungere gli obiettivi a più fasi. Presenta un loop specifico e ben strutturato che consente di gestire il ciclo di pensiero dell'AI. [4, 5]

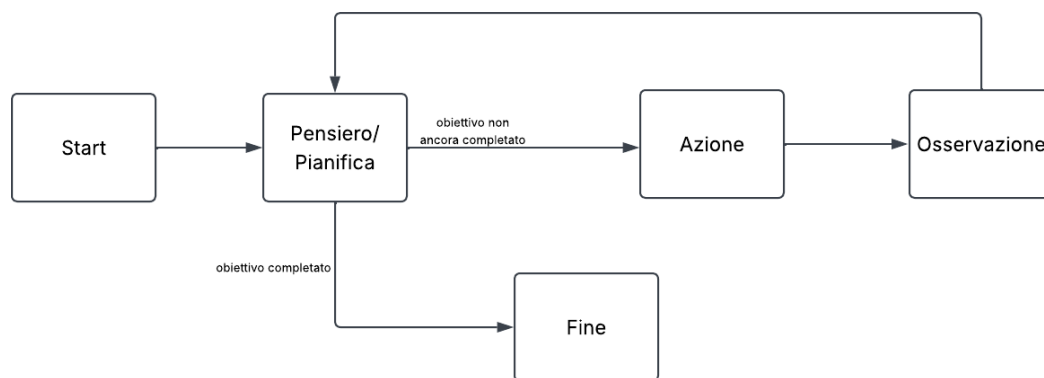


Figura 3.1: Flusso ReAct

Ci sono le tre fasi principali del flusso [4, 5]:

- **Pensiero/Pianifica (Thought):** Il modello ragiona su cosa sa, se ci sono ancora delle azioni da eseguire per completare un obiettivo o meno.
- **Azione (Action):** basando il pensiero, il modello decide ed esegue un'azione specifica utilizzando uno o più strumenti (*tool*) esterni messi a disposizione.
- **Osservazione (Observation):** Il modello riceve il risultato dell'azione e viene usato come nuovo contesto per il pensiero successivo.

Il ciclo continua finché il modello, tramite il proprio ragionamento, non ritiene di aver raggiunto una risposta finale.

3.2.2 Tool calling

Il *tool calling* è un meccanismo che permette a un'intelligenza artificiale di interagire con strumenti esterni come API, database o funzioni, attraverso un agente AI. [6]

Quando un modello AI non dispone di conoscenze sufficienti o ha bisogno di una funzione esterna per completare il task, in base ai strumenti disponibili nell'applicazione, può decidere di invocare uno o più tools (strumenti) per ottenere una risposta.

Per far sì che un LLM possa decidere autonomamente di invocare una o più funzioni, non è sufficiente inviare il messaggio dell'utente, è necessario anche fornire uno schema strutturato (in formato JSON) che descriva degli strumenti, con il nome, una descrizione e i parametri per ognuno di loro [7].

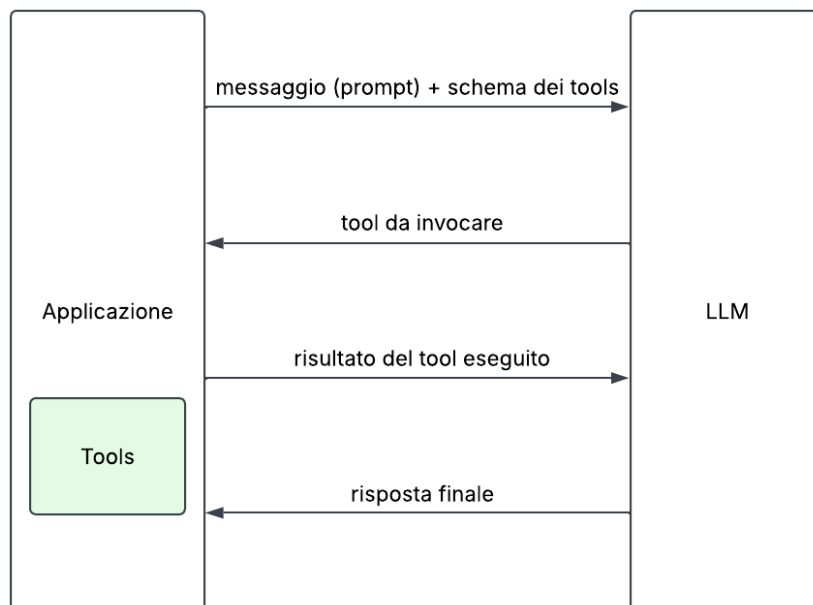


Figura 3.2: Flusso di un singolo tool calling

Il processo di invocazione di un singolo tool (Figura 3.2) è molto semplice. L'applicazione invia un messaggio e uno schema dei tools a un modello di intelligenza artificiale. Una volta elaborato, il modello manda un'informazione che specifica il tool da invocare all'applicazione. L'applicazione invoca il tool specificato e invia il risultato al modello AI per l'elaborazione di una risposta finale, che viene poi inviata all'applicazione. [7, 8] Function calling viene generalmente indicato come una forma di tool calling, in cui il modello richiede l'invocazione di una funzione definita dall'applicazione. In questo caso, il tool corrisponde a una porzione di codice eseguibile dall'applicazione. [6]

3.3 Sicurezza

Ci sono delle possibili superfici d'attacco da prendere in considerazione quando si sviluppa un agente AI e un'applicazione che permette di gestire il filesystem

3.3.1 Prompt Injection

Il *prompt injection* è una tecnica di attacco dove gli input sono costruiti apposta per manipolare il comportamento di un LLM, con l'obiettivo di forzare il modello a ignorare le istruzioni precedenti e ad eseguire istruzioni malevole o indesiderate [9].

3.3.1.1 Direct prompt injection

Il *direct prompt injection* avviene quando un utente fornisce direttamente al modello un input contenente le istruzioni che ne alterano il comportamento per aggirare le regole di sicurezza ed eseguire le istruzioni non previste. [9]

3.3.1.2 Indirect prompt injection

L'*indirect prompt injection* invece si verifica quando un LLM elabora contenuti provenienti da fonti esterne come pagine web, documenti o file, che contengono istruzioni appositamente progettate per manipolarne il comportamento. [9]

3.3.2 Path Traversal

L'attacco *path traversal* è una vulnerabilità che mira ad accedere a file e directory che si trovano al di fuori della directory autorizzata dall'applicazione. [10] Questa vulnerabilità avviene quando l'applicazione utilizza gli input messi dall'utente per costruire percorsi di filesystem senza effettuare i controlli e la validazione adeguati.

In questo progetto viene sviluppato un agente AI che interagisce con il filesystem all'interno di una directory assegnata, definita come *sandbox*. Pertanto è fondamentale che l'accesso dell'agente sia limitato a tale directory e alle relative sottodirectory. L'agente non deve in alcun modo accedere a risorse situate al di fuori di tale directory. Questo vincolo deve essere garantito effettuando controlli opportuni sul percorso richiesto.

3.3.3 Mitigazione

Per mitigare i rischi associati al *prompt injection* e al *path traversal*, è necessario che l'applicazione applichi controlli di sicurezza indipendentemente dal comportamento del modello AI. I contenuti forniti al modello, sia direttamente dall'utente sia indirettamente attraverso i file presenti nel filesystem, devono essere trattati con cautela, in quanto potrebbero contenere istruzioni malevole. Uno scenario possibile è quello in cui un file contenga istruzioni

che inducano l'LLM a tentare di accedere a risorse situate al di fuori della cartella autorizzata. Per questo motivo, per ogni operazione richiesta dall'LLM sul filesystem, il percorso deve essere verificato a livello del sistema, per assicurarsi che rimanga all'interno della *sandbox* e delle relative sottodirectory.

3.4 Human-in-the-loop

Human-in-the-loop è un meccanismo che permette agli utenti di partecipare attivamente alle decisioni e alle azioni dell'AI [11]. Invece di affidare completamente all'agente l'esecuzione delle azioni, il sistema può fermare il flusso e chiedere all'utente di verificare o approvare determinate operazioni. Questo approccio è molto utile quando un agente AI vuole eseguire le azioni distruttive o potenzialmente irreversibili, in questi casi il sistema ferma il flusso e chiede esplicitamente l'approvazione da parte dell'utente umano prima di procedere, riducendo così il rischio di operazioni indesiderate.

3.5 Google Gen AI SDK

Google Gen AI SDK è una libreria che permette di integrare i modelli di intelligenza artificiale di Google all'interno delle proprie applicazioni ed è disponibile per diversi linguaggi di programmazione, tra cui Java. Questa libreria offre diverse funzionalità, tra cui la generazione e l'analisi dei contenuti e la possibilità di utilizzare il function calling per invocare funzioni definite all'interno dell'applicazione.

3.6 Soluzioni esistenti

Esistono già strumenti basati su modelli di intelligenza artificiale in grado di interagire con il file system. Tra i principali esempi si possono citare Claude Code e OpenAI Codex, entrambi concepiti come agenti in grado di interpretare richieste complesse espresse in linguaggio naturale ed eseguirle sull'ambiente locale.

Claude Code e OpenAI Codex sono *coding agent* in grado di leggere e modificare file, nonché di eseguire comandi. Questi agenti possono analizzare i contenuti di un progetto e raggiungere gli obiettivi indicati dall'utente attraverso una sequenza di operazioni sul file system e sull'ambiente di esecuzione. In questo modo, l'interazione con il progetto non si limita alla generazione di codice, ma comprende anche l'analisi del contesto, la modifica dei file e l'esecuzione dei comandi necessari per portare a termine il compito richiesto [12, 13]. Per quanto riguarda i permessi e la sicurezza, Claude Code utilizza un sistema di gestione dei permessi per controllare l'utilizzo dei tool e l'accesso alle risorse del sistema. Per impostazione predefinita, le operazioni di sola lettura non richiedono generalmente l'approvazione dell'utente e possono, in determinate condizioni, accedere anche a risorse esterne

alla directory di lavoro. Per le operazioni con un impatto maggiore sul sistema, come la modifica dei file o l'esecuzione dei comandi tramite Bash, l'applicazione utilizza il sistema di approvazione e le relative regole di autorizzazione [14].

È possibile abilitare sandboxing, che introduce ulteriori restrizioni, limitando le risorse del file system e le operazioni accessibili a Claude Code. In questo modo è possibile ridurre la superficie di attacco e impedire che un agente possa accedere liberamente a risorse esterne al progetto. [14]

OpenAI Codex, invece, adotta un modello di sicurezza basato sulla combinazione di sandboxing e approvazione. Il sandbox definisce i limiti tecnici entro i quali l'agente può operare, mentre la policy di approvazione stabilisce quando Codex deve richiedere una conferma esplicita all'utente per eseguire un'operazione. Con la configurazione di default (*workspace-write*), Codex può leggere i file e modificare quelli presenti nella directory di lavoro e negli eventuali percorsi aggiuntivi autorizzati senza richiedere necessariamente un'approvazione per ogni operazione. Le operazioni che oltrepassano i limiti del sandbox, come la modifica di file al di fuori dei percorsi autorizzati, possono invece richiedere un'approvazione esplicita [15].

È possibile personalizzare il livello di autorizzazione di entrambi gli agenti in base alle esigenze dell'utente. È infatti possibile configurare gli agenti per permettere loro di operare in modo più autonomo, oppure adottare configurazioni maggiormente restrittive. Questo permette di bilanciare il livello di autonomia e di controllo, adattando il modello di sicurezza al contesto di utilizzo [14, 15].

Capitolo 4

Analisi

4.1 Requisiti funzionali e non funzionali

In questa sezione si presentano i requisiti funzionali e non funzionali fatti durante l'analisi di questo progetto.

4.1.1 Requisiti funzionali

ID	001
Nome	Configurazione progetto Spring Boot
Priorità	Alta
Descrizione	Il sistema deve essere inizializzato come applicazione Spring Boot con struttura multi-layer (controller, service, repository).

ID	002
Nome	Endpoint REST
Priorità	Alta
Descrizione	Il backend deve esporre endpoint REST per inviare messaggi all'agente, ricevere risposte e visualizzare in tempo reale il contenuto della sandbox.

ID	003
Nome	Servizio di gestione file con sandbox
Priorità	Alta
Descrizione	Tutte le operazioni devono essere eseguite da un sistema che opera esclusivamente all'interno di una directory sandbox.

ID	004
Nome	Validazione e blocco path traversal
Priorità	Alta
Descrizione	Il sistema deve rilevare e bloccare qualsiasi tentativo di path traversal prima di eseguire qualsiasi operazione su file.

ID	005
Nome	Annotazioni personalizzate per i tool
Priorità	Alta
Descrizione	Il sistema deve fornire un set di annotazioni per identificare i metodi invocabili dall'agente, specificando il nome, la descrizione e i parametri.

ID	006
Nome	Generazione automatica schema JSON
Priorità	Alta
Descrizione	Il sistema deve essere in grado di scansionare i tool e generare lo schema in JSON per l'API di Gemini.

ID	007
Nome	Invocazione dinamica dei tool via Reflection
Priorità	Alta
Descrizione	Il sistema deve poter permettere di invocare dinamicamente il metodo Java a runtime corrispondente al tool richiesto da un LLM.

ID	008
Nome	Ciclo ReAct
Priorità	Alta
Descrizione	Il sistema deve implementare il ciclo Reasoning-Acting iterativo.

ID	009
Nome	Gestione dello stato della conversazione
Priorità	Alta
Descrizione	Ogni sessione deve mantenere una finestra dei messaggi per garantire la coerenza del contesto durante le conversazioni.

ID	010
Nome	Interfaccia chat
Priorità	Alta
Descrizione	Il frontend deve mostrare la conversazione tra l'utente e l'agente.

ID	011
Nome	Visualizzazione file system sandbox
Priorità	Bassa
Descrizione	Il frontend deve mostrare la struttura ad albero aggiornata della sandbox con refresh automatico dopo ogni risposta dell'agente.

ID	012
Nome	Sistema Human-in-the-Loop per operazioni distruttive
Priorità	Alta
Descrizione	Prima di eseguire tool che modificano o eliminano file, il sistema deve sospendere l'esecuzione e chiedere conferma esplicita dell'utente.

4.1.2 Requisiti non funzionali

ID	013
Nome	Isolamento assoluto della sandbox
Priorità	Alta
Descrizione	Il sistema deve garantire che nessuna operazione sul file system possa avvenire al di fuori della directory sandbox, indipendentemente dall'input dell'AI o dell'utente.

ID	014
Nome	Autorizzazione human-in-the-loop per operazioni distruttive
Priorità	Alta
Descrizione	L'architettura deve rendere fisicamente impossibile eseguire operazioni distruttive senza la conferma esplicita dell'utente, indipendentemente dalla logica dell'AI.

ID	015
Nome	Gestione degli errori e resilienza
Priorità	Alta
Descrizione	Il sistema deve gestire gli errori (es. limiti token superati, timeout, ...) senza perdere lo stato della sessione.

ID	016
Nome	Estensibilità dei tool
Priorità	Alta
Descrizione	L'architettura deve essere costruita in modo estensibile, permettendo di aggiungere semplicemente nuovi tool senza modifiche al core del progetto.

ID	017
Nome	Logging strutturato
Priorità	Bassa
Descrizione	Il sistema deve produrre log strutturati per facilitare il debugging e il monitoraggio.

4.2 Use case

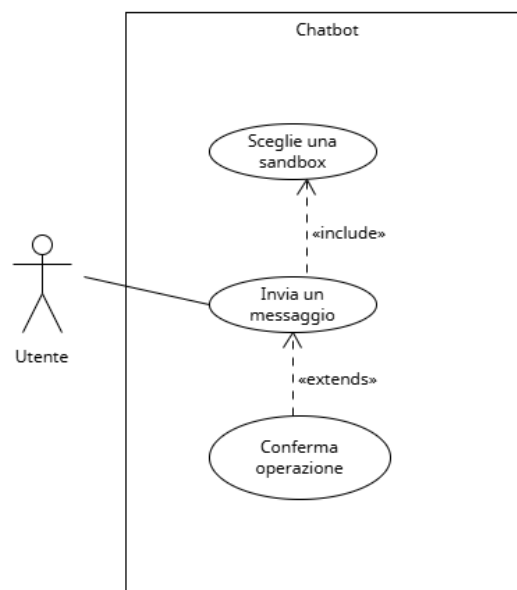


Figura 4.1: Diagramma use case

Il diagramma (vedi figura 4.1) mostra i casi d'uso del sistema, che prevede l'interazione dell'utente con il sistema stesso per l'invio di messaggi all'agente AI. L'utente può inviare messaggi all'agente AI, a condizione che sia già stato selezionato il percorso della cartella sandbox con cui il sistema deve interagire. Dopo aver inviato il messaggio, l'utente deve confermare o meno l'operazione distruttiva richiesta dal sistema, nel caso in cui l'agente AI volesse eseguirla.

Capitolo 5

Progettazione

5.1 Pianificazione

Dopo aver discusso con il committente e definito i requisiti del progetto, viene pianificato il lavoro per organizzare temporalmente le attività da fare.

Il seguente diagramma di Gantt (vedi figura 5.1) è stato realizzato per avere una visione chiara delle attività da svolgere. Ciò permette di monitorare lo stato di avanzamento del progetto.

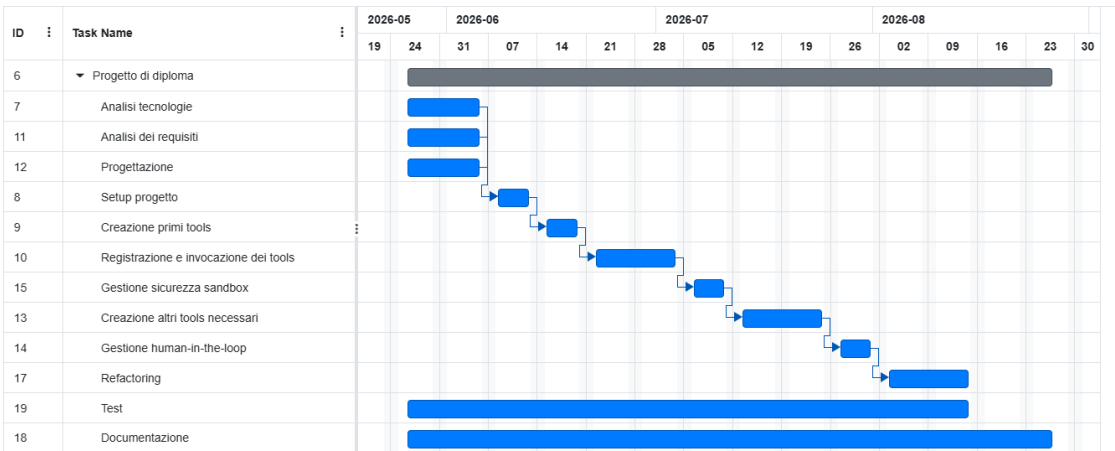


Figura 5.1: Gantt per la pianificazione temporale preventivo

5.2 Gestione del progetto

Durante il lavoro, si tengono incontri periodici con il committente per discutere i risultati ottenuti, ricevere feedback e pianificare i prossimi obiettivi, oltre a mostrare lo stato di avanzamento del progetto. Questo approccio consente di garantire che il progetto si concluda con un risultato finale che corrisponda alle aspettative del committente.

5.3 Tecnologie

5.3.1 Backend

Il linguaggio di programmazione utilizzato nel backend è Java, un linguaggio orientato agli oggetti e fortemente tipizzato. Le sue caratteristiche principali, che lo rendono noto, sono la portabilità e robustezza.

Spring Boot è un framework per lo sviluppo di applicazioni web in Java che permette di creare applicazioni in modo più rapido e strutturato, facilitando la gestione delle dipendenze e la realizzazione di API REST per permettere la comunicazione tra frontend e backend.

Viene utilizzato Maven, uno strumento per i progetti Java usato principalmente per gestire le dipendenze, compilare il codice, eseguire i test e creare i file distribuibili.

Nel backend viene inoltre utilizzata la libreria Google Gen AI SDK per interagire con i modelli Gemini di Google tramite API e per usare Function Calling, che permette di invocare le funzioni locali presenti nell'applicazione se necessario.

Le versioni delle tecnologie utilizzate per il backend sono le seguenti:

- Java: 21
- Spring Boot: 4.0.6
- Google Gen AI SDK: 1.61.0

5.3.2 Frontend

Le tecnologie utilizzate per il frontend sono HTML, CSS e JavaScript, che permettono di implementare un chatbot minimale e interattivo. HTML è stato usato per definire la struttura della pagina, CSS per personalizzare la presentazione grafica e JavaScript per implementare la logica lato client, rendere la pagina interattiva e comunicare con il backend. Sono state inoltre utilizzate le librerie Marked e DOMPurify. Marked serve a visualizzare correttamente il testo scritto in Markdown, mentre DOMPurify serve a sanitizzare il codice generato in HTML per prevenire l'inserimento di contenuti malevoli.

5.3.3 Version control

Il progetto viene salvato in una repository su GitLab, una piattaforma per la gestione e lo sviluppo di progetti software che utilizza Git, un sistema di controllo versione. GitLab integra inoltre funzionalità di CI/CD che permettono di automatizzare le fasi di compilazione e test del progetto prima della distribuzione.

5.4 Architettura

L'applicazione è strutturata secondo l'architettura mostrata nella figura 5.2, che mostra i vari componenti del sistema, ognuno con un compito specifico. Il frontend è il componente che interagisce con l'utente e il suo compito è visualizzare correttamente i dati raccolti dal backend e raccogliere gli input inseriti dall'utente. Il backend invece contiene la logica dell'applicazione e orchestra il flusso delle richieste. In caso di ricezione di un messaggio da parte dell'utente, il backend invia una richiesta a Gemini API e, se necessario, interagisce con il filesystem. Gemini API mette a disposizione dei modelli di intelligenza artificiale di Google per le applicazioni che ne fanno uso. Non è previsto l'utilizzo di un database, siccome in questo progetto non è necessaria la persistenza dei messaggi tra diverse sessioni. I dati della conversazione vengono mantenuti durante l'esecuzione dell'applicazione, ma si perdono alla sua chiusura.

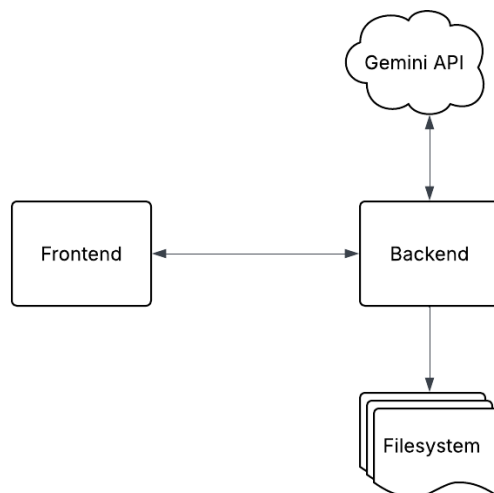


Figura 5.2: Architettura dell'applicativo

5.5 Struttura del progetto

5.5.1 Frontend

La struttura del codice nel frontend è molto semplice, c'è un file HTML `index.html` che contiene la struttura base della chat tra un utente e l'agente AI e ci sono le due cartelle `style` e `script` che contengono i file CSS e JavaScript rispettivamente.

5.5.2 Backend

La struttura del codice nel backend invece è più strutturata e robusta, seguendo l'architettura a strati per garantire la leggibilità, la manutenibilità e la testabilità. All'interno del package principale, il backend è suddiviso in:

- `controller`: contiene i controller che espongono le API REST dell'applicazione, gestendo le richieste HTTP provenienti dal frontend.
- `service`: contiene la logica applicativa del sistema, gestendo i messaggi con l'agente AI tramite il package `ai` e occupandosi della configurazione e della validazione del percorso della sandbox.
- `ai`: contiene la logica dell'AI. Implementa il meccanismo di function calling e la gestione di una sessione di chat stateful con un LLM, oltre alla logica di controllo del ciclo iterativo tra i comandi dell'AI e le funzioni locali presenti nell'applicazione.
- `tool`: contiene le funzioni necessarie per interagire con il filesystem e gestisce la sicurezza per evitare path traversal.
- `config`: contiene le configurazioni dell'applicazione.
- `exception`: gestisce gli errori dell'applicazione, definendo le eccezioni personalizzate e un handler globale che le intercetta e le traduce in risposte HTTP adeguate.
- `dto`: contiene gli oggetti di trasferimento di dati utilizzati per strutturare le risposte restituite dal backend al frontend.

5.6 Scelta del modello LLM di Gemini

Avendo il piano gratuito di Google AI Studio, ci sono diverse limitazioni sui modelli più performanti. La Tabella 5.1 riporta i limiti d'utilizzo dei modelli di testo disponibili nella versione gratuita.

Modello	RPM	TPM	RPD
Gemini 3.1 Flash Lite	15	250'000	500
Gemini 3.5 Flash	5	250'000	20
Gemini 3.5 Flash Lite	15	250'000	500
Gemini 3.6 Flash	5	250'000	20
Gemini 3.7 Flash	5	250'000	20

Tabella 5.1: Limiti dei modelli Gemini nel piano gratuito di Google AI Studio

Dove RPM indica il numero massimo di richieste al minuto, TPM il numero massimo di token di input al minuto e RPD il numero massimo di richieste giornaliere.

Sebbene Gemini 3.7 Flash sia il modello più performante tra quelli disponibili nel piano gratuito, la soglia massima di venti richieste al giorno risulta troppo restrittiva per l'applicazione finale.

Per questo motivo, si è scelto di utilizzare Gemini 3.5 Flash Lite, perché le sue capacità si rivelano sufficienti per gestire la struttura di un filesystem, operazione che non richiede un'elevata complessità di ragionamento ma trae grande vantaggio dal limite più elevato di richieste giornaliere (500 RPD).

5.7 Design delle interfacce

La pagina del chatbot è molto semplice, minimale e intuitiva. L'utente può interagire con l'agente AI inviando dei messaggi. Se l'agente AI dovesse voler eseguire un'operazione distruttiva, chiederebbe prima all'utente un'ulteriore conferma.

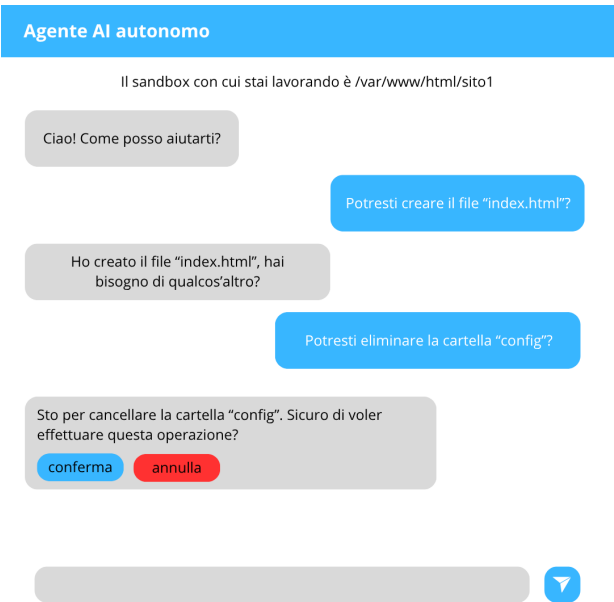


Figura 5.3: Mockup della chatbot

5.8 Diagrammi

5.8.1 Diagramma di flusso di gestione di function calling

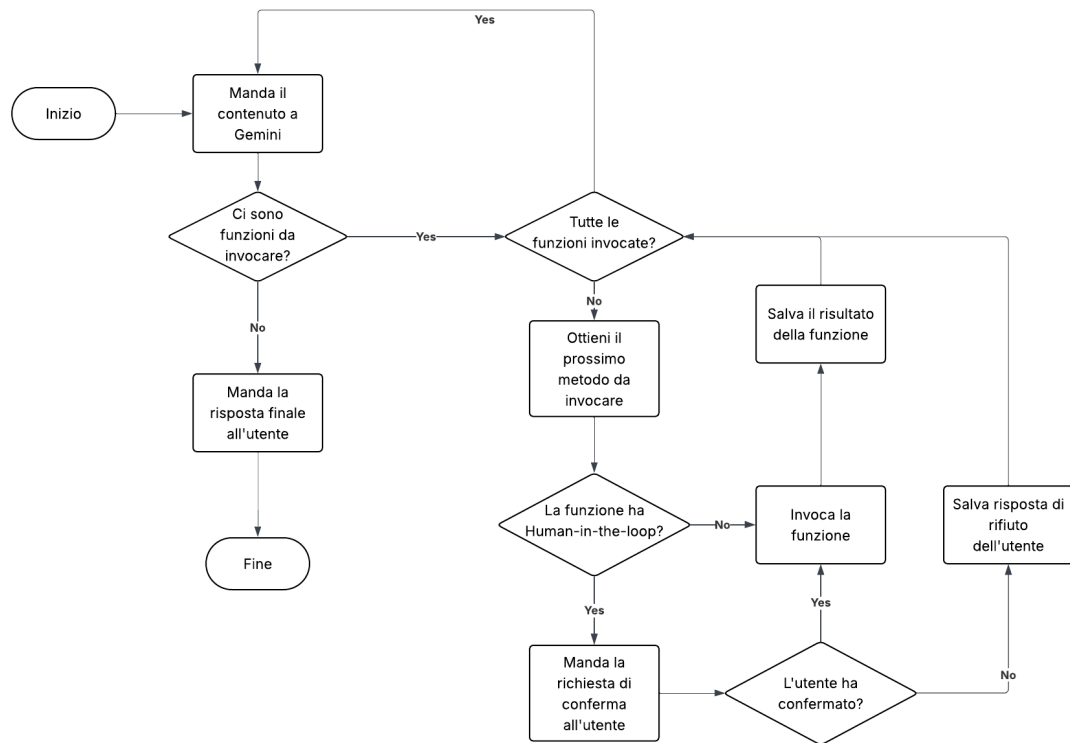


Figura 5.4: Diagramma di flusso di gestione di function calling

Il diagramma presentato nella figura 5.4 rappresenta il flusso delle operazioni eseguite dal sistema a seguito della ricezione di un prompt da parte dell'utente.

Inizialmente, il contenuto del prompt viene inviato al modello Gemini, che ne elabora le informazioni e restituisce una risposta. Il sistema verifica quindi se la risposta contiene una o più richieste di invocazioni di funzioni locali. Nel caso in cui non siano presenti funzioni da invocare, la risposta finale viene inoltrata direttamente all'utente e il flusso termina.

Qualora, invece, siano presenti una o più funzioni da invocare, queste vengono gestite sequenzialmente, una alla volta. Per ciascuna funzione viene verificato se sia necessaria una conferma da parte dell'utente. Se la funzione non richiede una conferma, essa viene invocata direttamente e il risultato ottenuto viene salvato. Nel caso in cui sia invece richiesta una conferma, il sistema invia una richiesta all'utente e attende la sua risposta. Se l'utente conferma l'operazione, la funzione viene invocata e il relativo risultato viene salvato. In caso contrario, il rifiuto dell'utente viene registrato, in modo da fornire successivamente tale informazione al modello Gemini e consentirgli di tenere conto del fatto che la funzione non è stata eseguita.

Una volta completata la gestione di tutte le funzioni richieste, i risultati delle invocazioni e gli eventuali rifiuti dell'utente vengono nuovamente forniti al modello Gemini. Il modello utilizza tali informazioni per elaborare la risposta successiva e, se necessario, pianificare ulteriori operazioni da eseguire. In questo modo, il processo può iterare fino a quando non sono più presenti funzioni da invocare e il sistema può restituire la risposta finale all'utente.

5.8.2 Diagramma di sequenza di un function calling

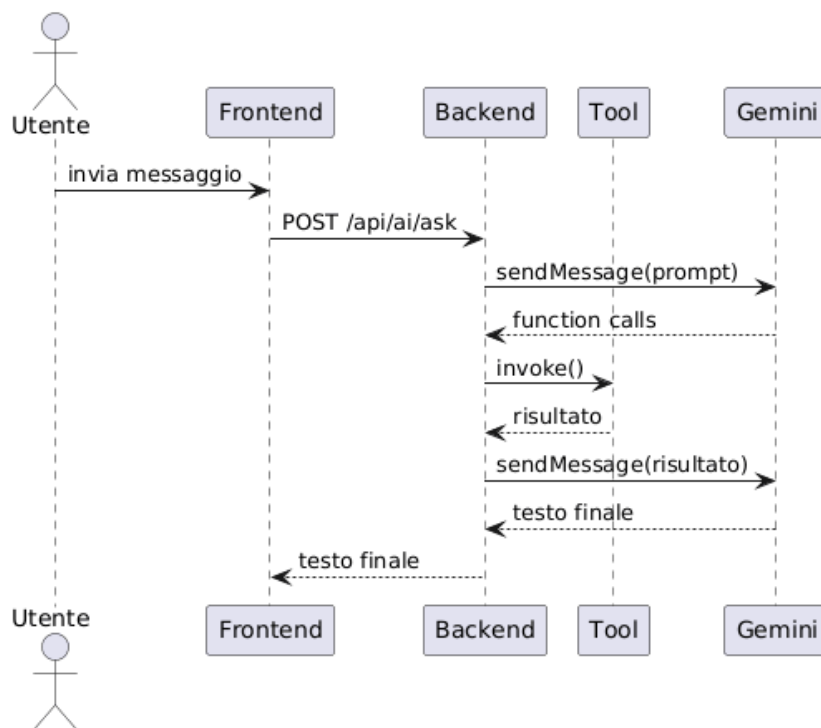


Figura 5.5: Diagramma di sequenza di un function call

Il diagramma presente nella figura 5.5 presenta il flusso di interazione temporale tra utente e vari componenti del sistema nel caso in cui il modello linguistico necessiti di invocare una funzione locale (function calling) per rispondere alla richiesta dell'utente.

L'utente invia un messaggio tramite il frontend, il quale inoltra la richiesta al backend attraverso una chiamata `POST /api/ai/ask`. Il backend, a sua volta, inoltra il prompt a Gemini tramite il metodo `sendMessage(prompt)` presente nella libreria di Google GenAI SDK. A questo punto, qualora il modello determini che per rispondere in modo adeguato sia necessario eseguire una sola azione specifica, Gemini restituisce al backend un function call, ovvero le indicazioni relative alla funzione da invocare e ai parametri da utilizzare. Il backend si occupa di invocare il tool corrispondente, il quale esegue l'operazione richiesta e restituisce il relativo risultato. Una volta ottenuto il risultato, il backend lo inoltra nuovamente a Gemini, in modo che il modello possa elaborare la risposta finale. Gemini restituisce quin-

di al backend il testo finale della risposta, che viene infine propagato al frontend e mostrato all'utente.

5.8.3 Diagramma di sequenza di un function calling con human in the loop

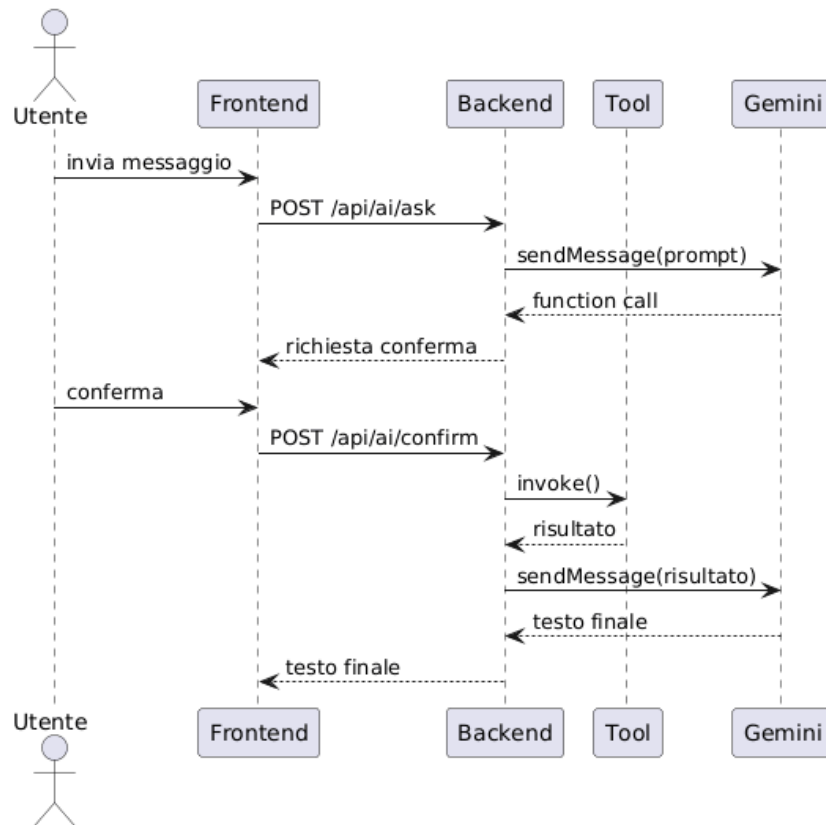


Figura 5.6: Diagramma di sequenza di un function call con human-in-the-loop accettando l'invocazione

Questo diagramma, presente nella figura 5.6, rispetto al diagramma nella figura 5.5, presenta un passaggio intermedio in più. Dopo che Gemini restituisce al backend una function call, se il sistema ritiene sia necessario ottenere una conferma esplicita da parte dell'utente, il backend invia al frontend una richiesta di conferma. Se l'utente intende procedere con l'esecuzione, conferma tramite il frontend, il quale inoltra la richiesta al backend attraverso POST /api/ai/confirm, permettendo così la ripresa del processo di function calling precedentemente interrotto.

Capitolo 6

Implementazione

6.1 Configurazione Spring Boot

Per la configurazione del progetto viene utilizzata Spring Initializr, che permette di creare e scaricare un progetto Spring già configurato. È stata selezionata Java 21 come versione di Java e Maven come sistema di build e gestione delle dipendenze. Inoltre, è stata aggiunta la dipendenza Spring Web, necessaria per la realizzazione degli endpoint REST e per la gestione delle richieste HTTP dell'applicazione.

Successivamente, nel file `application.properties` sono state aggiunte le seguenti proprietà dell'applicazione:

- `google.genai.api-key=${GOOGLE_GENAI_API_KEY}`, dove `GOOGLE_GENAI_API_KEY` è la variabile d'ambiente protetta e contiene una chiave API di Google AI Studio, in modo che l'applicazione possa interfacciarsi con i modelli Gemini di Google.
- `google.genai.model=gemini-3.5-flash-lite`, che specifica il modello Gemini 3.5 Flash Lite utilizzato.

6.2 Endpoint REST

Il backend espone le seguenti richieste API REST, ciascuna delle quali è associata a una funzione presente nelle classi di controller.

Metodo	URL	Descrizione
POST	/api/ai/ask	Invia un messaggio e ritorna una risposta finale dall'AI oppure una richiesta di conferma per eseguire un'operazione distruttiva da parte del sistema
POST	/api/ai/confirm	Conferma o meno un'operazione distruttiva sospesa dal sistema
GET	/api/sandbox	Ritorna il percorso della cartella sandbox
POST	/api/sandbox/new	Imposta il percorso della cartella sandbox

Tabella 6.1: API REST del backend

6.3 Service

Nel package `service` vengono implementate due classi: `GeminiService` e `SandboxService`. Il compito di `SandboxService` è verificare che il percorso della cartella indicato dall'utente esista, sia effettivamente una `directory` e disponga dei permessi necessari (lettura e scrittura) per l'esecuzione delle operazioni richieste. Una volta superati tali controlli, il percorso viene salvato come radice della `sandbox`.

`GeminiService`, invece, si occupa di inizializzare la sessione di chat dell'agente AI e di orchestrare, utilizzando le classi del package `ai`, i prompt inviati dall'utente e le relative conferme di esecuzione per i tool che richiedono un meccanismo di `human-in-the-loop`.

6.4 Sicurezza

È stata implementata la classe `PathValidator`, che tiene traccia della cartella `sandbox` e garantisce che ogni percorso richiesto dai tool dell'agente AI non possa accedere al di fuori di questa cartella. Viene implementato un metodo chiamato `resolveSandboxPath(Path pathInput)`, utilizzato da tutti i tool di `filesystem` prima di ogni operazione su disco. Il metodo esegue tre passaggi principali, oltre a verificare se la `sandbox` esiste ancora:

1. Utilizza il metodo `resolve` della classe `Path` per combinare il percorso richiesto con la radice della `sandbox`. Se il percorso fornito è relativo, viene concatenato alla cartella `sandbox`, altrimenti viene restituito direttamente il percorso fornito.
2. Utilizza il metodo `normalize` per risolvere eventuali `directory` speciali come `".."` e `"."`.

3. Utilizza il metodo `startsWith` per verificare che il percorso risultante si trovi ancora all'interno della cartella `sandbox`. Se la condizione è verificata, viene restituito il percorso risolto, altrimenti viene sollevata un'eccezione `SandboxViolationException`.

In questo modo viene rafforzato il confinamento tra la cartella `sandbox` e il resto del filesystem.

Questo approccio consente di eseguire i controlli a livello di sistema, indipendentemente dall'azione di un utente o di un LLM.

6.5 Tools

6.5.1 Annotation

Per far sì che i metodi presenti nell'applicazione possano essere invocati da un agente, sono state implementate le due annotazioni personalizzate `@AiTool` e `@AiToolParam`, applicabili rispettivamente a un metodo e ai suoi parametri. Queste annotazioni hanno un attributo chiamato `description` che permette di descrivere, rispettivamente, un metodo e i suoi parametri. In questo modo un LLM conosce i comportamenti dei metodi basandosi sulle descrizioni, poiché non ha accesso all'implementazione dei metodi.

Questo approccio serve a dichiarare le funzioni esposte al modello accanto alle loro implementazioni e consente al sistema di generare uno schema completo tramite reflection (approfondito nella sezione 6.6).

Viene inoltre implementata l'annotazione `@HumanInTheLoop`, applicabile ai metodi che, prima di essere eseguiti, richiedono una conferma esplicita da parte dell'utente. Il valore del suo attributo `description` corrisponde al messaggio di conferma mostrato all'utente.

6.5.2 Implementazione dei tools

Come menzionato nella sezione precedente, i metodi invocabili dall'agente AI utilizzano le annotazioni personalizzate.

Il codice seguente presenta un metodo annotato che permette di creare un file:

```
1 @AiTool(description="create a file at specified path")
2 public String createFile(@AiToolParam(description="Path of the file to be created"
3     ) String pathParameter) {
4     Path targetFile = Paths.get(pathParameter);
5     try {
6         targetFile = this.pathValidator.resolveSandboxPath(targetFile);
7     } catch (SandboxViolationException e) {
8         return String.format("Permission denied: %s", e.getMessage());
9     }
10
11     try {
12         Files.createFile(targetFile);
```

```
13     } catch (FileAlreadyExistsException e) {
14         return String.format("File '%s' already exists", pathParameter);
15     } catch (NoSuchFileException e) {
16         return String.format("Cannot create file: parent directory does not exist
17                               for '%s'", pathParameter);
18     } catch (IOException e) {
19         return "Something went wrong while creating the file";
20     }
21     return String.format("File '%s' created", pathParameter);
22 }
```

Come tutti gli altri metodi, la prima cosa che questo metodo fa è risolvere il percorso passato come parametro, interrompendo l'esecuzione nel caso in cui non vengano rispettati i requisiti di sicurezza. Una volta risolto il percorso, vengono eseguite le istruzioni specifiche per manipolare il filesystem, in questo caso la creazione del file. Infine, il metodo restituisce un messaggio che indica il risultato dell'azione all'LLM. È importante restituire anche i messaggi di errore, in modo che l'LLM possa sapere cosa è successo e adattare di conseguenza il proprio comportamento. Ovviamente si presta attenzione a non inviare i percorsi assoluti all'LLM, perché il modello deve conoscere solo la sandbox.

La seguente tabella presenta le implementazioni dei metodi invocabili dall'agente AI:

Nome metodo	parametri	Descrizione
createFile	percorso del file da creare	crea un file nel percorso specificato
createDirectory	percorso della directory da creare	crea una directory nel percorso specificato
readFile	percorso del file da leggere	legge il contenuto di un file nel percorso specificato
readFirstLines	percorso del file da leggere; numero di righe da leggere dall'inizio del file	legge le prime N righe di un file nel percorso specificato
readLastNLines	percorso del file da leggere; numero di righe da leggere dalla fine del file	legge le ultime N righe di un file nel percorso specificato
overwriteFile	percorso del file da scrivere; contenuto da scrivere	sovrascrive un file con il contenuto fornito, sostituendo il contenuto esistente
appendContent	percorso del file da scrivere; contenuto da scrivere	aggiunge del testo in coda a un file
insertAtLine	percorso del file da scrivere; numero di riga (a partire da 1); contenuto da inserire	inserisce una nuova riga di contenuto prima della riga specificata, spostando in basso le righe successive
deleteFileOrDirectory	percorso del file o della directory da eliminare	elimina un file o una directory vuota nel percorso specificato (richiede conferma dell'utente)
deleteDirectoryRecursively	percorso della directory da eliminare	elimina una directory e tutto il suo contenuto, anche se non vuota (richiede conferma dell'utente)
copyFile	percorso sorgente; percorso di destinazione	copia ricorsivamente un file o una directory da un percorso sorgente a un percorso di destinazione
moveFileOrDirectory	percorso sorgente; percorso di destinazione	sposta un file o una directory da un percorso sorgente a un percorso di destinazione
listDirectory	percorso della directory da elencare	elenca i file e le sottodirectory contenuti in una directory

Tabella 6.2: Metodi invocabili dall'agente AI

6.6 Scansione e registro dei tools

Per la registrazione automatica dei tool, vengono implementate le classi `ToolRegister` e `ToolDefinition` nel package `ai`. `ToolRegister` è un motore di scansione che analizza le classi contenenti i tool, individua i metodi annotati con `@AiTool` e genera automaticamente i relativi schemi per l'API di Gemini. `ToolDefinition` è invece la struttura dati che salva le informazioni necessarie su ciascun metodo registrato (istanza, riferimento al metodo e l'eventuale messaggio di human-in-the-loop) e ne permette l'invocazione dinamica a runtime tramite reflection.

`ToolRegister`, dopo aver individuato ogni metodo annotato con `@AiTool`, ne salva le informazioni in una nuova istanza di `ToolDefinition`, che viene poi inserita in una `HashMap` dove la chiave è il nome del metodo e il valore è l'istanza di `ToolDefinition` appena creata. Successivamente, una volta individuati tutti i metodi, per ciascuno di essi viene generato un oggetto `FunctionDeclaration` contenente nome, descrizione e schema dei parametri del metodo. Tutte le `FunctionDeclaration`, definite dalla libreria *Google GenAI SDK*, vengono poi raggruppate in un unico oggetto `Tool`, definito da quella libreria, che rappresenta l'insieme dei tool che il modello può utilizzare per generare una risposta.

6.7 Invocazione a runtime

Nella classe `ToolDefinition` viene implementato un metodo che permette di invocare dinamicamente un metodo salvato in quell'istanza a runtime tramite Java Reflection, passando i parametri ricevuti dal modello sotto forma di una `Map<String, Object>`, dove la chiave è il nome del parametro e il valore è l'argomento fornito dal modello.

Prima di invocare un metodo, gli argomenti vengono salvati in un array ordinato `Object[]` coerente con la firma del metodo originale, in modo che possa essere invocato correttamente. Una volta invocato il metodo, il risultato viene restituito. Se un metodo invocato lancia un'eccezione, questa viene propagata come `InvocationTargetException`, così da poter essere gestita da chi ha richiesto l'invocazione.

6.8 Configurazione della sessione chat stateful

Viene implementata la classe `ChatClient` che gestisce una sessione di chat tra l'utente e l'agente AI.

Quando viene creata una sessione, registra i tool tramite `ToolRegister` e ne ricava le `FunctionDeclaration`.

Successivamente, viene configurato `ThinkingConfig`, una classe del Google GenAI SDK, che configura la capacità del modello di generare un ragionamento interno, utile per monitorare il pensiero dell'AI. Viene impostato il livello di ragionamento medio, per trovare un

compromesso tra la profondità del pensiero generato e i tempi e i costi della risposta. Viene inoltre richiesta l'inclusione nella risposta dei passaggi del ragionamento interno, se il modello li supporta.

Infine, all'interno della classe `ChatClient` viene creata un'istanza di `Chat`, alla quale vengono passate le configurazioni e i tool. `Chat` è un oggetto fornito dalla libreria Google GenAI SDK che gestisce i messaggi e mantiene lo stato della conversazione tra le interazioni successive, evitando di doverne gestire manualmente lo stato all'interno dell'applicazione.

6.9 Meccanismo di function calling

La classe `ChatClient` implementa il meccanismo ReAct per eseguire i metodi Java, osservarne il risultato e proseguire il ragionamento.

Dopo aver inviato un prompt, la risposta del modello viene analizzata per verificare la presenza di richieste di function calling. In assenza di richieste di questo tipo, la risposta testuale viene restituita direttamente al frontend. In presenza di richieste, invece, il controllo passa al ciclo di esecuzione.

Le function call richieste dal modello vengono processate in modo sequenziale. Per ognuna, il sistema verifica se necessita di conferma da parte dell'utente e, in tal caso, il ciclo di esecuzione viene sospeso, lo stato viene salvato in un oggetto `FunctionPending` (implementato nel package `ai`), che contiene la function call corrente, i risultati già raccolti e le function call rimanenti, e all'utente viene richiesta conferma esplicita prima di procedere. Se invece non è richiesta conferma, il metodo viene eseguito tramite reflection e il risultato dell'invocazione, o l'eventuale eccezione sollevata, viene incapsulato in un oggetto `FunctionResponse`, a sua volta racchiuso in un oggetto `Part` (entrambi implementati nella libreria Google GenAI SDK). Una volta esaurita la coda delle function call richieste, le istanze `Part` vengono inviate al modello in un unico messaggio, permettendogli di proseguire il ragionamento su tutti i risultati osservati. Il ciclo viene ripetuto finché il modello non necessita più di ulteriori function call, e la risposta finale viene mostrata all'utente.

6.10 Gestione degli errori

6.10.1 HTTP

Il sistema implementa una gestione degli errori a livello HTTP per i casi in cui l'utente fornisce input non validi oppure l'applicazione non è configurata correttamente, ad esempio path malformati o chiave dell'API esterna non valida.

Viene implementato il `GlobalControllerExceptionHandler`, una classe che centralizza la traduzione delle eccezioni in risposte HTTP standard, intercettando:

- `BadRequestException`: eccezione personalizzata sollevata in presenza di errori di validazione applicativa, ad esempio dal `SandboxService` quando il path della sandbox non esiste, non è una directory, oppure il sistema non ha i permessi di lettura o scrittura necessari;
- `InvalidPathException`: eccezione sollevata quando il path fornito ha una sintassi non valida per il sistema operativo sottostante;
- `ClientException` (della libreria Google GenAI SDK): eccezione sollevata quando la chiamata all'API di Gemini fallisce. Il gestore analizza il messaggio restituito dall'API per distinguere le cause più comuni, tra cui la chiave API non valida, permessi insufficienti o quota esaurita, restituendo un messaggio descrittivo e un codice di errore applicativo specifico per ciascun caso.

6.10.2 Livello agentico

Le eccezioni lanciate dai metodi invocati dall'agente AI vanno gestite in modo che il flusso agentico non venga interrotto.

Se un metodo invocato via reflection solleva un'eccezione non catturata internamente, il sistema intercetta `InvocationTargetException` e `IllegalAccessException` e le converte in una `Part` contenente un `FunctionResponse` con il tipo di errore e il relativo messaggio, che viene poi inviata al modello. In questo modo un errore imprevisto di un tool non interrompe il ciclo conversazionale, ma diventa un risultato osservabile in base al quale il modello può adattare il proprio ragionamento, ad esempio riprovando con parametri diversi o spiegando il problema all'utente.

6.11 Logging

Per monitorare il comportamento dell'agente AI, è stato integrato un sistema di logging usando la libreria `slf4j`, già fornita di default da Spring Boot. Il logging viene utilizzato come strumento per rendere tracciabile il ciclo di ragionamento dell'agente AI, che segue il pattern ReAct. Ogni fase del ciclo viene registrata a livello `INFO`:

- **Thought**: il ragionamento intermedio generato dal modello, utile per capire perché l'agente ha scelto una determinata azione;
- **Act**: l'invocazione di un tool, con il nome della funzione richiesta dal modello;
- **Observe**: il risultato restituito dal tool dopo l'esecuzione, che viene poi reinserito nel contesto della conversazione.

Inoltre, il sistema di logging registra anche il meccanismo di human-in-the-loop, così da rendere tracciabili gli esiti delle decisioni dell'utente prima dell'esecuzione delle azioni distruttive.

6.12 Frontend

È stato implementato un frontend minimale, composto da HTML, CSS e JavaScript vanilla, servito staticamente da Spring Boot. Il frontend realizza un'interfaccia di messaggistica per la conversazione tra l'utente e l'agente AI.

All'avvio dell'applicazione, viene mostrato un modal che chiede all'utente di specificare la cartella su cui l'agente dovrà operare. Il percorso inserito viene inviato tramite `POST /api/sandbox/new`, dopodiché il modal viene chiuso e l'interfaccia di chat diventa utilizzabile.

Una volta configurata la sandbox, l'utente può digitare un prompt e inviarlo all'endpoint `POST /api/ai/ask`. La risposta del backend include un campo `status`, corrispondente all'enumerazione `ChatStatus` presente nel backend, che determina come il messaggio viene renderizzato:

- **COMPLETED**: il messaggio dell'assistente viene mostrato come risposta definitiva alla richiesta dell'utente;
- **CONFIRMATION_NEEDED**: viene invece mostrato un messaggio di conferma con due pulsanti, *Conferma* e *Annulla*, che permettono all'utente di autorizzare o negare l'esecuzione di un'azione distruttiva. La scelta viene inviata all'endpoint `POST /api/ai/confirm` e la conversazione riprende sulla base della decisione dell'utente, riflettendo lato client il meccanismo di human-in-the-loop descritto in precedenza.

I messaggi dell'agente AI vengono renderizzati come Markdown tramite la libreria *marked*, e successivamente sanitizzati con *DOMPurify* prima di essere inseriti nel DOM. Questo passaggio è necessario poiché il testo proviene dall'output di un modello linguistico, e non può quindi essere considerato input fidato. La sanitizzazione previene l'esecuzione di eventuali attacchi XSS. I messaggi dell'utente, al contrario, vengono inseriti come semplice testo, senza interpretazione dell'HTML.

Capitolo 7

Test

7.1 Metodologia di testing

Durante lo sviluppo viene implementata una suite di test unitari per verificare sia la correttezza funzionale delle operazioni sul filesystem, sia la robustezza della sandbox contro input malevoli, in particolare attacchi di tipo Path Traversal, al fine di garantirne la sicurezza e l'isolamento.

7.2 Strumenti utilizzati

Il framework utilizzato per il testing automatico in Java è *JUnit 5*. Viene inoltre utilizzato *Mockito* per simulare le dipendenze esterne e controllare il comportamento delle funzioni testate. Per i test che coinvolgono operazioni sul filesystem, viene utilizzato un filesystem temporaneo reale e isolato, tramite l'annotazione `@TempDir` di JUnit, grazie alla quale ogni test dispone di una propria directory, ripulita automaticamente al termine dell'esecuzione.

7.3 Unit testing

La classe `PathValidator` e le classi dei tool sono state testate in quanto esposte all'agente AI per l'interazione con il filesystem, ovvero il punto più critico dal punto di vista della sicurezza e della correttezza funzionale. Per ogni classe vengono testati sistematicamente il caso di successo, il caso di risorsa non trovata, i conflitti tra risorse (ad esempio destinazione già esistente) e la violazione della sandbox.

7.4 Risultati dei test

Vengono eseguite sette classi di test, per un totale di sessantuno test unitari. Tutti i test sono stati eseguiti con esito positivo.

7.5 Pipeline CI/CD

Viene utilizzata GitLab CI/CD per automatizzare il ciclo di vita del software, garantendo che ogni modifica al codice sorgente venga testata e correttamente integrata, fino a raggiungere uno stato pronto per il rilascio in produzione.

7.5.1 Struttura della pipeline

La pipeline è suddivisa in tre stage:

- **Build:** il codice sorgente viene compilato tramite `mvn compile`, fornendo un primo feedback rapido su eventuali errori di compilazione prima di procedere con l'esecuzione dei test.
- **Test:** viene eseguita l'intera suite di test unitari tramite `mvn test`. I risultati vengono esportati in formato JUnit XML e pubblicati come report della pipeline, permettendo a GitLab di mostrarne direttamente l'esito nell'interfaccia della merge request.
- **Package:** viene generato il file JAR dell'applicazione tramite `mvn package -DskipTests`. Questo stage viene eseguito esclusivamente sui branch `main` e `dev`, evitando di produrre artefatti per gli altri branch di lavoro.

Capitolo 8

Conclusioni

8.1 Risultati

Il progetto ha raggiunto gli obiettivi prefissati e sono stati soddisfatti i requisiti stabiliti durante la fase di progettazione. L'agente AI sviluppato è ora in grado di eseguire autonomamente operazioni su un file system a partire da istruzioni scritte in linguaggio naturale. Il sistema può eseguire le operazioni di base per la gestione del file system, come la lettura, la scrittura, lo spostamento, la copia e l'eliminazione di file e cartelle. Prima di eseguire un'operazione potenzialmente pericolosa, viene inoltre richiesta esplicitamente la conferma dell'utente. Grazie all'implementazione del pattern ReAct, il sistema è in grado di eseguire più istruzioni in modo iterativo, al fine di raggiungere obiettivi complessi. L'applicazione integra inoltre un sistema di logging che consente di monitorare il ciclo di pensiero dell'AI. Infine, il sistema è stato progettato per garantire la sicurezza e la robustezza, prevenendo gli attacchi di tipo Path Traversal da parte dell'AI, limitando le operazioni dell'agente esclusivamente a una directory di lavoro e impedendo l'accesso alle risorse al di fuori di tale directory.

8.2 Sviluppi futuri

È possibile migliorare il sistema e arricchirlo con nuove funzionalità.

8.2.1 Nuovi tool

Estensione dei tool con nuove funzionalità, tra cui la creazione di link simbolici, l'elenco ricorsivo delle cartelle, la ricerca di un pattern testuale all'interno dei file (grep) e l'esecuzione di comandi shell, senza dover modificare il core del progetto.

8.2.2 Conversazioni persistenti

Creazione di un sistema persistente che permetta di salvare le conversazioni scambiate tra l'utente e l'agente AI, per poterle riutilizzare in un secondo momento. A tale scopo,

è possibile utilizzare Spring Data JPA per far interfacciare Spring Boot con un database relazionale. Questo permetterebbe all'utente di riprendere una conversazione terminata in precedenza.

8.2.3 Supporto multi-provider LLM

Supportare più provider LLM oltre a Gemini, tra cui OpenAI, Anthropic e anche modelli locali tramite Ollama. Questo permetterebbe di scegliere tra modelli di provider diversi e di non dipendere da un singolo provider. L'utilizzo di modelli locali tramite Ollama garantisce inoltre la privacy, poiché i dati non vengono esposti all'esterno, pur riducendo le prestazioni. L'utente può quindi decidere se affidarsi ai provider esterni oppure ai modelli in locale, in base alle proprie necessità. Dal punto di vista implementativo, sarebbe necessario introdurre un'astrazione che disaccoppi `ChatClient` dal SDK specifico di Google GenAI, così da poter aggiungere nuove implementazioni per ciascun provider.

8.2.4 Streaming delle risposte in tempo reale

Utilizzare le tecnologie SSE oppure WebSocket per lo streaming delle risposte in tempo reale, mostrando anche il ragionamento iterativo del pattern ReAct. Attualmente il client riceve un'unica risposta solo al termine dell'intero ciclo ReAct, mentre i pensieri intermedi dell'agente vengono registrati esclusivamente nei log lato server. Lo streaming permetterebbe invece di mostrare all'utente, in tempo reale, ogni fase di ragionamento, migliorando la trasparenza.

Bibliografia

- [1] Cole Stryker. What are large language models (llms)? IBM. [Online]. Disponibile: <https://www.ibm.com/it-it/think/topics/large-language-models>. Accesso: 4-Ago-2026.
- [2] Databricks. What is agentic ai? Databricks. [Online]. Disponibile: <https://www.databricks.com/it/blog/what-is-agentic-ai>. Accesso: 4-Ago-2026.
- [3] Anna Gutowska. What are ai agents? IBM. [Online]. Disponibile: <https://www.ibm.com/think/topics/ai-agents>. Accesso: 4-Ago-2026.
- [4] Dave Bargmann. What is a react agent? IBM. [Online]. Disponibile: <https://www.ibm.com/think/topics/react-agent>. Accesso: 6-Ago-2026.
- [5] Dian Yu Nan Du Izhak Shafran Karthik Narasimhan Yuan Cao Shunyu Yao, Jeffrey YZ-hao. React: Synergizing reasoning and acting in language models. Department of Computer Science, Princeton University and Google Research, Brain team. [Online]. Disponibile: <https://react-lm.github.io/>. Accesso: 6-Ago-2026.
- [6] Databricks. What is tool calling? Databricks. [Online]. Disponibile: <https://www.databricks.com/blog/what-is-tool-calling>. Accesso: 10-Ago-2026.
- [7] Google. Introduction to function calling. Google. [Online]. Disponibile: <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/tools/function-calling>. Accesso: 15-Giu-2026.
- [8] Cole Stryker. What is tool calling? IBM. [Online]. Disponibile: <https://www.ibm.com/think/topics/tool-calling>. Accesso: 10-Ago-2026.
- [9] OWASP. Llm01:2025. OWASP. [Online]. Disponibile: <https://genai.owasp.org/llmrisk/llm01-prompt-injection>. Accesso: 16-Ago-2026.
- [10] OWASP. Path traversal. OWASP. [Online]. Disponibile: <https://owasp.org/www-community/attacks/PathTraversal>. Accesso: 16 – Ago – 2026.
- [11] Cole Stryker. What is human-in-the-loop? IBM. [Online]. Disponibile: <https://www.ibm.com/think/topics/human-in-the-loop>. Accesso: 12-Ago-2026.

- [12] Anthropic. Come funziona claude code. Anthropic. [Online]. Disponibile: <https://code.claude.com/docs/it/how-claude-code-works>, Accesso: 25-Ago-2026.
- [13] OpenAI. Ti presentiamo codex. OpenAI. [Online]. Disponibile: <https://openai.com/it-IT/index/introducing-codex/>, Accesso: 25-Ago-2026.
- [14] Anthropic. Claude code - configure permission. Anthropic. [Online]. Disponibile: <https://code.claude.com/docs/en/permissions>, Accesso: 25-Ago-2026.
- [15] OpenAI. Utilizzare codex in sicurezza in openai. OpenAI. [Online]. Disponibile: <https://openai.com/it-IT/index/running-codex-safely>, Accesso: 25-Ago-2026.